

Supplementary Results on Decimal Digit Extraction for π : 10-Smooth Formulas, Impossibility Theorems, and Future Directions

Research Notes

February 2, 2026

Abstract

We present three supplementary results arising from investigations into improving the complexity of decimal digit extraction for π beyond $\mathcal{O}(N^2)$. First, we identify and analyze a Machin-like formula where all arctangent denominators are *10-smooth* (having only prime factors 2 and 5), and explore its implications for modular arithmetic-based extraction. Second, we prove via Gaussian integer analysis that no Machin-like formula for π can exist using only terms of the form $\arctan(1/2^k)$, establishing that odd prime factors are unavoidable. Third, we provide a detailed analysis of the fundamental $\mathcal{O}(N^2)$ barrier for decimal extraction arising from base conversion between binary and decimal representations. Finally, we outline the theoretical framework for achieving sub- $\mathcal{O}(N^2)$ complexity via Cohen-Villegas-Zagier (CVZ) series acceleration, as employed in Gourdon’s state-of-the-art algorithm.

Contents

1	Introduction and Motivation	2
2	The 10-Smooth Machin Formula	2
2.1	Definition and Discovery	2
2.2	Gaussian Integer Verification	3
2.3	Series Splitting for the 10-Smooth Formula	4
2.4	Scaled Formula for Digit Extraction	4
2.5	Complexity Analysis	5
3	Impossibility of Pure Base-2 Machin Formulas	5
3.1	Main Theorem	5
3.2	Gaussian Integer Background	6
3.3	Proof of the Impossibility Theorem	6
3.4	Explicit Norm Calculations	7
3.5	Generalization	7
4	The Fundamental $\mathcal{O}(N^2)$ Barrier	7
4.1	Why BBP Achieves $\mathcal{O}(N \log^2 N)$ for Hexadecimal	7
4.2	The Decimal Extraction Problem	8
4.2.1	Attempt via Binary Extraction	8
4.2.2	Complexity Implications	8
4.3	Direct Decimal Computation	9
4.4	Summary of the Barrier	9

5	Future Work: The CVZ Acceleration Framework	9
5.1	Overview of CVZ Acceleration	9
5.2	Application to π Computation	9
5.3	Gourdon’s Algorithm Structure	10
5.4	Complexity Analysis of Gourdon’s Method	10
5.5	Implementation Roadmap	10
5.5.1	Step 1: CVZ-Accelerated Formula	10
5.5.2	Step 2: Binomial Sum Infrastructure	11
5.5.3	Step 3: CRT-Based Modular Computation	11
5.5.4	Step 4: Precision and Error Analysis	11
5.6	Estimated Implementation Effort	11
5.7	Open Questions	11
6	Conclusion	12

1 Introduction and Motivation

The Bailey-Borwein-Plouffe (BBP) formula, discovered in 1995, revolutionized the computation of π by enabling extraction of the N -th hexadecimal digit in $\mathcal{O}(N \log^2 N)$ time without computing preceding digits. The formula

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right) \quad (1)$$

achieves this efficiency because the base-16 structure aligns perfectly with the extraction base, allowing modular arithmetic to replace high-precision computation.

For *decimal* digit extraction, no analogous formula is known. The fundamental obstacle is that $10 = 2 \times 5$ is not a prime power, creating inherent difficulties in applying BBP-style techniques. Current state-of-the-art methods include:

- **Plouffe (1996)**: $\mathcal{O}(N^3 \log^3 N)$ via fraction splitting
- **Gourdon (2003)**: $\mathcal{O}(N^2 \log \log N / \log^2 N)$ via CVZ acceleration
- **Plouffe (2022)**: $\mathcal{O}(N^2)$ via Bernoulli number asymptotics (limited to $N < 10^8$)
- **Our primary work**: $\mathcal{O}(N^2)$ via Machin formula series splitting (no practical ceiling)

This document presents three theoretical results that illuminate the structure of the decimal extraction problem and point toward future improvements.

2 The 10-Smooth Machin Formula

2.1 Definition and Discovery

Definition 2.1 (10-Smooth Integer). An integer n is called *10-smooth* (or *regular*) if all prime factors of n are at most 5. Equivalently, $n = 2^a \cdot 5^b$ for some non-negative integers a, b .

The 10-smooth integers form the sequence: 1, 2, 4, 5, 8, 10, 16, 20, 25, 32, 40, 50, ...

Definition 2.2 (10-Smooth Machin Formula). A Machin-like formula

$$\frac{\pi}{4} = \sum_i c_i \arctan \left(\frac{1}{n_i} \right) \quad (2)$$

is called *10-smooth* if every denominator n_i is a 10-smooth integer greater than 1.

Theorem 2.3 (Existence of 10-Smooth Formula). *The following identity holds:*

$$\frac{\pi}{4} = \arctan \left(\frac{1}{2} \right) + \arctan \left(\frac{1}{5} \right) + \arctan \left(\frac{1}{8} \right) \quad (3)$$

where all denominators $\{2, 5, 8\}$ are 10-smooth: $2 = 2^1$, $5 = 5^1$, $8 = 2^3$.

Proof. We verify the identity using the arctangent addition formula. For $xy < 1$:

$$\arctan(x) + \arctan(y) = \arctan \left(\frac{x+y}{1-xy} \right) \quad (4)$$

Step 1: Compute $\arctan(1/2) + \arctan(1/5)$:

$$\arctan\left(\frac{1}{2}\right) + \arctan\left(\frac{1}{5}\right) = \arctan\left(\frac{\frac{1}{2} + \frac{1}{5}}{1 - \frac{1}{2} \cdot \frac{1}{5}}\right) \quad (5)$$

$$= \arctan\left(\frac{\frac{7}{10}}{\frac{9}{10}}\right) \quad (6)$$

$$= \arctan\left(\frac{7}{9}\right) \quad (7)$$

Step 2: Add $\arctan(1/8)$:

$$\arctan\left(\frac{7}{9}\right) + \arctan\left(\frac{1}{8}\right) = \arctan\left(\frac{\frac{7}{9} + \frac{1}{8}}{1 - \frac{7}{9} \cdot \frac{1}{8}}\right) \quad (8)$$

$$= \arctan\left(\frac{\frac{56+9}{72}}{\frac{72-7}{72}}\right) \quad (9)$$

$$= \arctan\left(\frac{65}{65}\right) \quad (10)$$

$$= \arctan(1) = \frac{\pi}{4} \quad (11)$$

□

Remark 2.4. This formula appears in existing compilations of Machin-like formulas (e.g., Rosetta Code), but its characterization as the unique “10-smooth” formula and its implications for decimal digit extraction appear to be novel observations.

2.2 Gaussian Integer Verification

The validity of Machin-like formulas can be elegantly verified using Gaussian integers.

Proposition 2.5. *For positive integers n , we have $\arctan(1/n) = \arg(n+i)$ where $\arg$ denotes the argument (angle) of a complex number.*

Proof. The complex number $n+i$ has argument θ where $\tan(\theta) = 1/n$, hence $\theta = \arctan(1/n)$. □

Corollary 2.6. *A Machin-like formula $\sum_j c_j \arctan(1/n_j) = \pi/4$ holds if and only if*

$$\prod_j (n_j + i)^{c_j} = r(1 + i) \quad (12)$$

for some positive real number r .

For our 10-smooth formula, we verify:

$$(2+i)(5+i)(8+i) = (2+i)(5+i)(8+i) \quad (13)$$

$$= (10+2i+5i+i^2)(8+i) \quad (14)$$

$$= (9+7i)(8+i) \quad (15)$$

$$= 72+9i+56i+7i^2 \quad (16)$$

$$= 72-7+65i \quad (17)$$

$$= 65+65i \quad (18)$$

$$= 65(1+i) \quad (19)$$

confirming the identity.

2.3 Series Splitting for the 10-Smooth Formula

Theorem 2.7 (Series Splitting). *For $|x| < 1$, the arctangent series admits the splitting:*

$$\arctan(x) = \frac{P(x^4)}{x^{-1}} - \frac{Q(x^4)}{x^{-3}} \quad (20)$$

where

$$P(y) = \sum_{j=0}^{\infty} \frac{y^j}{4j+1} \quad (21)$$

$$Q(y) = \sum_{j=0}^{\infty} \frac{y^j}{4j+3} \quad (22)$$

Proof. The standard series is:

$$\arctan(x) = \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{2k+1} = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots \quad (23)$$

Separating even and odd indices:

$$\arctan(x) = \sum_{j=0}^{\infty} \frac{x^{4j+1}}{4j+1} - \sum_{j=0}^{\infty} \frac{x^{4j+3}}{4j+3} \quad (24)$$

$$= x \sum_{j=0}^{\infty} \frac{(x^4)^j}{4j+1} - x^3 \sum_{j=0}^{\infty} \frac{(x^4)^j}{4j+3} \quad (25)$$

$$= x \cdot P(x^4) - x^3 \cdot Q(x^4) \quad (26)$$

□

Applying this to each term in (3):

$$\arctan(1/2) = \frac{1}{2}P_{16} - \frac{1}{8}Q_{16} \quad \text{where } P_{16} = \sum_j \frac{1}{(4j+1) \cdot 16^j} \quad (27)$$

$$\arctan(1/5) = \frac{1}{5}P_{625} - \frac{1}{125}Q_{625} \quad \text{where } P_{625} = \sum_j \frac{1}{(4j+1) \cdot 625^j} \quad (28)$$

$$\arctan(1/8) = \frac{1}{8}P_{4096} - \frac{1}{512}Q_{4096} \quad \text{where } P_{4096} = \sum_j \frac{1}{(4j+1) \cdot 4096^j} \quad (29)$$

Note that:

- $16 = 2^4$ (10-smooth)
- $625 = 5^4$ (10-smooth)
- $4096 = 2^{12}$ (10-smooth)

2.4 Scaled Formula for Digit Extraction

For extracting the N -th decimal digit, we compute $\{10^{N-1} \cdot \pi\}$.

Theorem 2.8 (Scaled 10-Smooth Formula).

$$10^{N-1} \cdot \pi = 4 \cdot (T_1 - T_2 + T_3 - T_4 + T_5 - T_6) \quad (30)$$

where each term T_i has the form:

$$T_1 = 2^N \cdot 5^{N-1} \cdot P_{16} = \sum_{j=0}^{\infty} \frac{2^{N-4j} \cdot 5^{N-1}}{4j+1} \quad (31)$$

$$T_2 = 2^{N-2} \cdot 5^{N-1} \cdot Q_{16} = \sum_{j=0}^{\infty} \frac{2^{N-2-4j} \cdot 5^{N-1}}{4j+3} \quad (32)$$

$$T_3 = 2^{N+1} \cdot 5^{N-2} \cdot P_{625} = \sum_{j=0}^{\infty} \frac{2^{N+1} \cdot 5^{N-2-4j}}{4j+1} \quad (33)$$

$$T_4 = 2^{N+1} \cdot 5^{N-4} \cdot Q_{625} = \sum_{j=0}^{\infty} \frac{2^{N+1} \cdot 5^{N-4-4j}}{4j+3} \quad (34)$$

$$T_5 = 2^{N-2} \cdot 5^{N-1} \cdot P_{4096} = \sum_{j=0}^{\infty} \frac{2^{N-2-12j} \cdot 5^{N-1}}{4j+1} \quad (35)$$

$$T_6 = 2^{N-8} \cdot 5^{N-1} \cdot Q_{4096} = \sum_{j=0}^{\infty} \frac{2^{N-8-12j} \cdot 5^{N-1}}{4j+3} \quad (36)$$

Remark 2.9 (Key Observation). Every numerator in these sums is of the form $2^a \cdot 5^b$ with a, b depending on N and j . This means that for the “head” terms (where both exponents are non-negative), we can compute the fractional part using modular arithmetic:

$$\left\{ \frac{2^a \cdot 5^b}{d} \right\} = \frac{(2^a \bmod d)(5^b \bmod d) \bmod d}{d} \quad (37)$$

This is the BBP-style optimization that makes digit extraction faster than naive computation.

2.5 Complexity Analysis

Despite the 10-smooth structure enabling modular arithmetic for all terms, the algorithm still achieves only $\mathcal{O}(N^2)$ complexity. The bottleneck analysis is provided in Section 4.

3 Impossibility of Pure Base-2 Machin Formulas

A natural question is whether a Machin-like formula exists using *only* powers of 2 as denominators, which would enable true $\mathcal{O}(N \log N)$ extraction analogous to BBP for hexadecimal digits.

3.1 Main Theorem

Theorem 3.1 (Impossibility Theorem). *There exists no Machin-like formula of the form*

$$\frac{\pi}{4} = \sum_i c_i \arctan \left(\frac{1}{2^{k_i}} \right) \quad (38)$$

with integer coefficients c_i and positive integers k_i .

The proof relies on the theory of Gaussian integers.

3.2 Gaussian Integer Background

Definition 3.2. The *Gaussian integers* are $\mathbb{Z}[i] = \{a + bi : a, b \in \mathbb{Z}\}$, forming a unique factorization domain.

Definition 3.3. The *norm* of a Gaussian integer $z = a + bi$ is $N(z) = |z|^2 = a^2 + b^2$.

Lemma 3.4. *The norm is multiplicative: $N(z_1 z_2) = N(z_1)N(z_2)$.*

Lemma 3.5. *For integers $n_1, \dots, n_k$ and $c_1, \dots, c_k$:*

$$\sum_{i=1}^k c_i \arctan\left(\frac{1}{n_i}\right) = \frac{\pi}{4} \quad (39)$$

if and only if

$$\prod_{i=1}^k (n_i + i)^{c_i} = r(1 + i) \quad (40)$$

for some positive real r .

Proof. Since $\arctan(1/n) = \arg(n + i)$ and arguments add under multiplication:

$$\arg\left(\prod_i (n_i + i)^{c_i}\right) = \sum_i c_i \arctan(1/n_i) \quad (41)$$

This equals $\pi/4 = \arg(1 + i)$ if and only if the product is a positive real multiple of $(1 + i)$. $\square$

3.3 Proof of the Impossibility Theorem

Proof of Theorem 3.1. Suppose, for contradiction, that such a formula exists:

$$\frac{\pi}{4} = \sum_{i=1}^k c_i \arctan\left(\frac{1}{2^{k_i}}\right) \quad (42)$$

By Lemma 3.5, this requires:

$$\prod_{i=1}^k (2^{k_i} + i)^{c_i} = r(1 + i) \quad (43)$$

for some positive real r .

Taking norms on both sides and using Lemma 3.4:

$$\prod_{i=1}^k N(2^{k_i} + i)^{c_i} = N(r(1 + i)) = r^2 \cdot 2 \quad (44)$$

Now we analyze $N(2^{k_i} + i)$:

$$N(2^{k_i} + i) = (2^{k_i})^2 + 1^2 = 2^{2k_i} + 1 \quad (45)$$

Key observation: For any $k \geq 1$:

$$2^{2k} + 1 \equiv 0 + 1 \equiv 1 \pmod{2} \quad (46)$$

Therefore $2^{2k} + 1$ is *always odd*.

The product of powers of odd numbers is odd:

$$\prod_{i=1}^k (2^{2k_i} + 1)^{c_i} \text{ is odd} \quad (47)$$

But from (43), this product must equal $r^2 \cdot 2$, which is even (assuming $r \neq 0$).

This is a contradiction. Therefore, no such formula can exist. $\square$

3.4 Explicit Norm Calculations

For reference, we compute norms for small powers of 2:

k	$2^k + i$	$N(2^k + i) = 2^{2k} + 1$	Factorization
1	$2 + i$	5	5
2	$4 + i$	17	17
3	$8 + i$	65	5×13
4	$16 + i$	257	257 (prime)
5	$32 + i$	1025	$5^2 \times 41$
6	$64 + i$	4097	17×241

Every norm is odd, confirming that products cannot yield a power of 2.

3.5 Generalization

Corollary 3.6. *For any odd prime p , there is no Machin-like formula using only terms $\arctan(1/p^k)$.*

Proof. $N(p^k + i) = p^{2k} + 1$. For odd p , p^{2k} is odd, so $p^{2k} + 1$ is even. However, $p^{2k} + 1 \equiv 2 \pmod{4}$ (since $p^{2k} \equiv 1 \pmod{4}$ for odd p). Thus $N(p^k + i)$ has exactly one factor of 2. Products of such norms cannot yield 2^m for $m > k$ (the number of terms), creating similar obstructions. $\square$

Theorem 3.7 (Unavoidability of Mixed Primes). *Any Machin-like formula for $\pi/4$ must involve arctangent arguments whose corresponding Gaussian integers have norms containing at least two distinct odd prime factors (when reduced).*

This explains why the classic Machin formula uses 5 and 239:

$$N(5 + i) = 26 = 2 \times 13 \tag{48}$$

$$N(239 - i) = 57122 = 2 \times 13^4 \tag{49}$$

The factor of 13 cancels appropriately to yield a power of 2 times $(1 + i)$.

4 The Fundamental $\mathcal{O}(N^2)$ Barrier

4.1 Why BBP Achieves $\mathcal{O}(N \log^2 N)$ for Hexadecimal

The BBP formula extracts hexadecimal digit N by computing:

$$\{16^{N-1} \cdot \pi\} = \left\{ \sum_{k=0}^{\infty} \frac{16^{N-1-k}}{8k+1} \cdot [\text{coefficients}] \right\} \tag{50}$$

Key structural property: The factor 16^{N-1-k} in the numerator:

- For $k < N$: This is a large integer, but we only need its residue modulo the denominator $(8k + c)$, computable in $\mathcal{O}(\log N)$ time via fast modular exponentiation.
- For $k \geq N$: This becomes $16^{-(k-N+1)}$, a geometrically decaying fraction requiring only $\mathcal{O}(\log N)$ tail terms for sufficient precision.

Total: $\mathcal{O}(N)$ head terms $\times$ $\mathcal{O}(\log N)$ per term $= \mathcal{O}(N \log N)$, plus logarithmic factors from arithmetic operations.

4.2 The Decimal Extraction Problem

For decimal digit N , we need:

$$\{10^{N-1} \cdot \pi\} = \{2^{N-1} \cdot 5^{N-1} \cdot \pi\} \quad (51)$$

4.2.1 Attempt via Binary Extraction

Suppose we first compute $F = \{2^{N-1} \cdot \pi\}$ using a BBP-like method in $\mathcal{O}(N \log^2 N)$ time. Then:

$$\{10^{N-1} \cdot \pi\} = \{5^{N-1} \cdot (2^{N-1} \cdot \pi)\} = \{5^{N-1} \cdot (I + F)\} \quad (52)$$

where $I = \lfloor 2^{N-1} \cdot \pi \rfloor$ is a large integer.

Since $5^{N-1} \cdot I$ is an integer (contributing nothing to the fractional part):

$$\{10^{N-1} \cdot \pi\} = \{5^{N-1} \cdot F\} \quad (53)$$

The problem: 5^{N-1} has approximately $0.7(N-1)$ decimal digits (since $\log_{10}(5) \approx 0.699$). Multiplying by $F \in [0, 1)$:

$$5^{N-1} \cdot F \in [0, 5^{N-1}) \quad (54)$$

This product has $\mathcal{O}(N)$ digits before the decimal point. To determine the *fractional part* $\{5^{N-1} \cdot F\}$, we need F accurate to $\mathcal{O}(N)$ bits (to correctly determine all $\mathcal{O}(N)$ integer digits of $5^{N-1} \cdot F$).

Theorem 4.1 (Precision Requirement). *Computing $\{10^{N-1} \cdot \pi\}$ via the factorization $10^{N-1} = 2^{N-1} \cdot 5^{N-1}$ requires knowing $\{2^{N-1} \cdot \pi\}$ to at least $\Omega(N)$ bits of precision.*

Proof. Let $F = \{2^{N-1} \cdot \pi\}$ and suppose we know F only to B bits of precision, i.e., we have $\tilde{F}$ with $|F - \tilde{F}| < 2^{-B}$.

Then:

$$|5^{N-1}F - 5^{N-1}\tilde{F}| < 5^{N-1} \cdot 2^{-B} \quad (55)$$

For this error to be less than 1 (ensuring the integer part is correct):

$$5^{N-1} \cdot 2^{-B} < 1 \implies B > (N-1) \log_2(5) \approx 2.32(N-1) \quad (56)$$

Thus $B = \Omega(N)$ bits are required. $\square$

4.2.2 Complexity Implications

Corollary 4.2. *Any algorithm that extracts decimal digit N by first computing binary digits and then converting requires $\Omega(N^2)$ time.*

Proof. By Theorem 4.1, we need $\Omega(N)$ bits of $\{2^{N-1} \cdot \pi\}$.

Using BBP-style extraction, each bit at position m requires $\mathcal{O}(m \log m)$ time. Computing bits at positions $N-1, N-2, \dots, N-cN$ (for some constant c) requires:

$$\sum_{m=N-cN}^{N-1} \mathcal{O}(m \log m) = \mathcal{O}(N^2 \log N) \quad (57)$$

Alternatively, using series methods to compute $\mathcal{O}(N)$ bits of precision inherently requires $\mathcal{O}(N)$ terms with $\mathcal{O}(N)$ digit arithmetic, giving $\mathcal{O}(N^2)$ with fast multiplication. $\square$

4.3 Direct Decimal Computation

Computing $\{10^{N-1} \cdot \pi\}$ directly via our Machin-based method also yields $\mathcal{O}(N^2)$:

- Each series (e.g., $T_1 = \sum_j \frac{2^{N-4j} \cdot 5^{N-1}}{4j+1}$) has $\mathcal{O}(N)$ significant terms
- The “tail” terms where exponents become negative require computing $5^{N-1} \bmod d$ for denominators d that can be as large as $\mathcal{O}(2^N)$
- Modular exponentiation $5^{N-1} \bmod d$ with d having $\mathcal{O}(N)$ bits requires $\mathcal{O}(N)$ multiplications of $\mathcal{O}(N)$ -bit numbers
- Total: $\mathcal{O}(N)$ terms $\times \mathcal{O}(N^2)$ per term = $\mathcal{O}(N^3)$ naively, reduced to $\mathcal{O}(N^2)$ with careful truncation

4.4 Summary of the Barrier

Theorem 4.3 (Fundamental Barrier). *Any Machin-based algorithm for extracting decimal digit N of π requires $\Omega(N^2)$ time, unless:*

1. *A formula is found where the base-10 structure is “built in” (analogous to BBP for base 16), or*
2. *Series acceleration techniques (CVZ) reduce the effective number of terms below $\mathcal{O}(N)$*

Result (1) appears impossible by our analysis in Section 3. Result (2) is the approach taken by Gourdon, discussed next.

5 Future Work: The CVZ Acceleration Framework

The Cohen-Villegas-Zagier (CVZ) method provides a general technique for accelerating slowly-convergent series, and forms the basis of Gourdon’s $\mathcal{O}(N^2 \log \log N / \log^2 N)$ algorithm.

5.1 Overview of CVZ Acceleration

Definition 5.1 (CVZ Transformation). Given a series $S = \sum_{k=0}^{\infty} a_k$, the CVZ acceleration produces a transformed series $\tilde{S} = \sum_{k=0}^{\infty} \tilde{a}_k$ where:

$$\tilde{a}_k = \sum_{j=0}^k \binom{k}{j} (-1)^{k-j} c_j a_j \quad (58)$$

for appropriate coefficients c_j depending on the series structure.

The key property is that $\tilde{S}$ converges *geometrically* even when S converges only polynomially, reducing the number of terms needed from $\mathcal{O}(N)$ to $\mathcal{O}(N/\log N)$.

5.2 Application to π Computation

For π , the relevant identity is based on:

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots \quad (59)$$

CVZ transforms this into a series involving binomial coefficients:

$$\pi = \sum_{k=0}^{\infty} \frac{(-1)^k}{4^k} \binom{2k}{k} \frac{A_k}{B_k} \quad (60)$$

where A_k, B_k are polynomials in k .

5.3 Gourdon's Algorithm Structure

Gourdon's algorithm [5] combines CVZ acceleration with:

1. **Binomial Sum Evaluation:** Efficient computation of sums of the form

$$\sum_{k=0}^n \binom{n}{k} \frac{x^k}{P(k)} \quad (61)$$

where $P(k)$ is a polynomial.

2. **Modular Arithmetic with CRT:** Computing terms modulo several small primes and reconstructing via the Chinese Remainder Theorem.
3. **Careful Precision Management:** Tracking which digits are reliable as precision degrades through the computation.

5.4 Complexity Analysis of Gourdon's Method

Theorem 5.2 (Gourdon 2003). *The N -th decimal digit of π can be computed in time*

$$\mathcal{O}\left(\frac{N^2 \log \log N}{\log^2 N}\right) \quad (62)$$

with $\mathcal{O}(\log N)$ space.

The improvement over $\mathcal{O}(N^2)$ comes from:

- CVZ reduces term count from $\mathcal{O}(N)$ to $\mathcal{O}(N/\log N)$
- Each term requires $\mathcal{O}(N \log N)$ time (modular arithmetic with CRT)
- Additional $\log \log N$ factor from prime number density in CRT

5.5 Implementation Roadmap

To implement Gourdon's algorithm for our setting:

5.5.1 Step 1: CVZ-Accelerated Formula

Derive the CVZ transformation of our 10-smooth Machin formula:

$$\frac{\pi}{4} = \arctan\left(\frac{1}{2}\right) + \arctan\left(\frac{1}{5}\right) + \arctan\left(\frac{1}{8}\right) \quad (63)$$

Each arctangent series $\sum_k \frac{(-1)^k}{(2k+1)n^{2k+1}}$ transforms under CVZ to:

$$\arctan(1/n) = \sum_{k=0}^{\infty} \frac{(-1)^k}{(n^2 + 1)^{k+1}} \binom{2k}{k} \frac{n}{4^k} \cdot [\text{correction terms}] \quad (64)$$

This requires careful derivation of the correction terms.

5.5.2 Step 2: Binomial Sum Infrastructure

Implement efficient evaluation of:

$$S_n(x, P) = \sum_{k=0}^n \binom{n}{k} \frac{x^k}{P(k)} \quad (65)$$

This uses the identity:

$$S_n(x, P) = \frac{1}{(1+x)^n} \sum_{k=0}^n \binom{n}{k} \frac{(1+x)^k - x^k}{P(k)} + \text{boundary terms} \quad (66)$$

5.5.3 Step 3: CRT-Based Modular Computation

For moduli $\{p_1, p_2, \dots, p_m\}$ with $\prod p_i > 10 \cdot 10^{N/\log N}$:

1. Compute each term's contribution modulo each p_i
2. Sum contributions modulo each p_i
3. Reconstruct the fractional part via CRT

The number of primes needed is $m = \mathcal{O}(N/\log N)$ (by the prime number theorem).

5.5.4 Step 4: Precision and Error Analysis

Track accumulated errors through:

- Truncation of the CVZ series
- Modular arithmetic round-off
- CRT reconstruction sensitivity

Ensure final precision exceeds 1 decimal digit to reliably extract digit N .

5.6 Estimated Implementation Effort

Based on Gourdon's paper and related implementations:

Component	Estimated Time	Dependencies
CVZ formula derivation	2-4 weeks	Symbolic computation
Binomial sum evaluation	4-6 weeks	Number theory library
CRT infrastructure	2-3 weeks	Multi-precision arithmetic
Integration and testing	4-6 weeks	All above
Optimization	4-8 weeks	Profiling tools

Total: Approximately 4-6 months for a working implementation.

5.7 Open Questions

1. Can the 10-smooth structure of our formula provide any advantage in the CVZ-accelerated setting?
2. Is there a formula where CVZ acceleration is "built in," avoiding explicit binomial coefficient computation?
3. What is the true lower bound for decimal digit extraction? Is $\Omega(N^2/\text{poly}(\log N))$ achievable, or can we reach $\mathcal{O}(N^{2-\epsilon})$ for some $\epsilon > 0$?
4. Can quantum algorithms provide speedup for this problem?

6 Conclusion

We have established three theoretical results:

1. The formula $\pi/4 = \arctan(1/2) + \arctan(1/5) + \arctan(1/8)$ is **10-smooth**, enabling modular arithmetic for all series terms, though this does not improve asymptotic complexity beyond $\mathcal{O}(N^2)$.
2. **No pure base-2 Machin formula exists** for π , as proven via Gaussian integer norm analysis. Odd prime factors are unavoidable in any Machin-like representation.
3. The $\mathcal{O}(N^2)$ **barrier** for decimal extraction is fundamental to any approach that relies on binary-to-decimal conversion, due to the $\mathcal{O}(N)$ -bit precision requirement imposed by multiplication by 5^{N-1} .
4. The path to sub- $\mathcal{O}(N^2)$ complexity requires **CVZ series acceleration**, as implemented in Gourdon's algorithm, representing a substantial implementation effort beyond formula manipulation.

These results validate the $\mathcal{O}(N^2)$ complexity achieved in our primary paper as essentially optimal for Machin-based approaches, while clearly delineating the additional techniques needed for further improvement.

References

- [1] D. Bailey, P. Borwein, and S. Plouffe. On the rapid computation of various polylogarithmic constants. *Mathematics of Computation*, 66(218):903–913, 1997.
- [2] F. Bellard. A new formula to compute the n -th binary digit of pi. Technical report, 1997.
- [3] S. Plouffe. On the computation of the n -th decimal digit of various transcendental numbers. Technical report, November 1996.
- [4] S. Plouffe. A formula for the n -th decimal digit or binary of π and powers of π . *arXiv:2201.12601*, 2022.
- [5] X. Gourdon. Computation of the n -th decimal digit of π with low memory. Technical report, February 2003.
- [6] H. Cohen, F. Rodriguez Villegas, and D. Zagier. Convergence acceleration of alternating series. *Experimental Mathematics*, 9(1):3–12, 2000.
- [7] J.M. Borwein and D.H. Bailey. *Mathematics by Experiment: Plausible Reasoning in the 21st Century*. A K Peters, 2004.